

Supplementary Material: RepairWitness—Certifying Actionable Disagreements in Security Advisories

Anonymous Author(s)

Abstract—This supplement gives the complete semantic contract, formal arguments, certificate schemas, benchmark construction, validation campaigns, and clean-room reproduction procedure for RepairWitness. It expands the proof sketches in the main paper and records the exact frozen evidence behind every reported result.

I. COMPLETE REPAIR-ACTION SEMANTICS

A. Normalized claim and release-universe objects

A normalized claim retains source lineage, record identifier, source locator and member digest, ecosystem-qualified package identity, vulnerability aliases, affected-range events, explicit affected releases, fixed or patched anchors, alternative mitigations, withdrawal state, and a deterministic claim identifier. Source-local fields are preserved for audit but excluded from structural-equality payloads. Package normalization never erases ecosystem identity.

A release universe is an ordered tuple $U_p = (v_1, \dots, v_n)$ bound to a package key, registry endpoint family, response receipt, canonicalization policy, and SHA-256 digest. Every attempted package acquisition has one terminal status: SUCCESS, EMPTY, or FAILED. Empty and failed rows are evidence, not missing data to be silently removed.

B. Action constructors

For claim c and installed release $v \in U_p$, the evaluator returns one tagged action:

$$\mathcal{A}(c, v) \in \{\text{NO_ACTION}, \text{UPGRADE}(T), \text{ALTERNATIVE}(M), \text{REPAIR_NO_TARGET}, \text{UNKNOWN}(B)\}.$$

T is a nonempty canonical set of releases in U_p ; M is a nonempty normalized mitigation description; and B is a typed blocker. Constructor validation enforces disjoint fields: an unknown result cannot carry targets, a concrete result cannot carry a blocker, and an empty upgrade cannot masquerade as repair without a public target.

Affectedness is evaluated before target realization. Explicit affected versions use exact membership. OSV-style event sequences preserve multiple introduced/fixed/last-affected/limit segments. Ecosystem requirements are dispatched to their native or conservative parser. Unsupported syntax, ordering failure, withdrawal ambiguity, missing package identity, or a nonterminal universe produces UNKNOWN rather than a guessed Boolean.

For an affected release and anchor t , target realization is

$$\rho(t, U_p, v) = \min_{\prec} \{x \in U_p : x \models t \wedge v \prec x\},$$

TABLE I
FROZEN VERSION AND REQUIREMENT SEMANTICS.

Ecosystem	Ordering	Accepted requirement forms
PyPI	PEP 440	specifier conjunctions, epochs, pre/post/dev releases
npm	SemVer	comparisons, hyphen, caret, tilde, wildcard, Boolean joins
Cargo	Cargo semver	bare, caret, tilde, comparison, and comma joins
Go	module semver	comparison bounds; pseudo-versions only when natively parsed
NuGet	NuGet	bracket intervals, exact bounds, and floating ranges
Hex	Hex semver	comparisons and pessimistic requirements
Packagist	Composer	comparison, caret, tilde, wildcard, OR/AND forms
RubyGems	Gem version	comparison conjunctions and pessimistic operator
Maven	Maven order	interval unions and exact boundaries

when all operations are supported. Exact anchors are eligible only when present in U_p ; requirement anchors are filtered over U_p . The resulting action means that the source supports the target as a repair anchor. It is not a claim of compatibility, maintained status, exploitability reduction, or global safety.

Lemma 1 (Public-target invariant). *Every target in an accepted upgrade action belongs to the digest-bound release universe used to evaluate the claim.*

Proof. Exact anchors are accepted only by membership in U_p . Requirement anchors are realized by filtering elements of U_p . No other target constructor exists. $\square$

C. Ecosystem dispatch and abstention matrix

Version identifiers are interpreted only after ecosystem dispatch. The implementation never sends an arbitrary string through a single universal comparator. The supported surface is deliberately narrower than every native package-manager language, because an unsupported expression must be visible as uncertainty rather than silently approximated.

The frozen conformance suite contains accepted, rejected, boundary, prerelease, normalization, and cross-implementation cases for each ecosystem. It tests, among other cases, PEP 440 epochs and local segments; SemVer prerelease ordering and ignored build metadata; Cargo’s interpretation of a bare version as caret-compatible; Go’s leading-version marker; NuGet’s distinct lower-bound and exact syntax; Hex and Composer pessimistic upper bounds; RubyGems prereleases; and Maven

TABLE II
STRUCTURAL ADAPTER OUTPUTS RETAINED PER RECORD.

Field class	Examples	Purpose
Identity	source, member, aliases	grouping and traceability
Package	ecosystem, name, package URL	release-universe lookup
Affectedness	events, exact releases, requirements	partial Boolean trace
Repair evidence	fixed anchors, patched ranges, mitigations	action construction
Status	withdrawn flag, unsupported form, reason	typed abstention
Integrity	member and projection digests	deterministic reconstruction

interval unions. A requirement parser returns three pieces of evidence: parse status, normalized comparator object, and the source expression. The source text is retained even when evaluation abstains.

A release identifier is never coerced into another ecosystem merely because the target parser accepts it. For example, a Maven qualifier is not ordered with PEP 440, a Go pseudo-version is not decomposed as ordinary SemVer unless the Go implementation accepts it, and a NuGet bare version is not interpreted as an equality predicate. These rules prevent seemingly innocuous normalization from changing the repair action.

D. Source-specific adapter contract

The repository contains adapters for OSV JSON, PyPA OSV-compatible YAML, Go OSV JSON, RustSec Markdown/TOML, and RubySec YAML, plus registry adapters for nine package ecosystems. The source side produces a common structural claim but does not erase representation-specific evidence.

OSV records may contain several affected package entries; each becomes a package-local claim with a stable index in its identifier. RustSec and RubySec patched requirements are retained as requirements rather than collapsed to one guessed version. Git commit ranges remain source evidence but are non-executable in a release-only universe unless a separate reachability relation is supplied. Alternative mitigations are admitted only when they are explicit and normalizable; generic prose such as “upgrade recommended” does not create a mechanism action.

Grouping is package-scoped. Claims become adjacent when their normalized vulnerability aliases intersect, and groups are connected components so transitive alias closure is explicit. Primary claim-pair edges compare distinct curation lineages; same-lineage duplicates may be retained for provenance but do not become independent votes. Canonical endpoint order makes edge identifiers independent of file traversal order.

E. Edge disposition

For claim edge $e = (c_i, c_j)$, define the concrete domain and witness set

$$C_e = \{v \in U_p : \mathcal{A}(c_i, v), \mathcal{A}(c_j, v) \neq \text{UNKNOWN}\},$$

$$W_e = \{v \in C_e : \mathcal{A}(c_i, v) \neq \mathcal{A}(c_j, v)\}.$$

An executable edge is RESOLVED-DIVERGENT when $W_e \neq \emptyset$, RESOLVED-EQUIVALENT when $C_e = U_p$ and $W_e = \emptyset$, and INDETERMINATE otherwise. A failed or empty universe is NON-EXECUTABLE before these rules are applied.

Proposition 1 (Unknown cannot create a witness). *Replacing either action at a release by UNKNOWN cannot add that release to W_e .*

Proof. Membership in W_e requires both actions to be concrete. The replacement removes that antecedent. $\square$

The implementation also emits a transition label for each concrete witness: target disagreement, boundary disagreement, mechanism disagreement, or public-target availability disagreement. These labels are recomputed from action objects and are not inferred from advisory prose.

II. OPTIMIZATION PROBLEM AND GENERAL CERTIFICATES

A. Weighted redundant action separation

For divergent edges E , costs $w : U_p \rightarrow \mathbb{Z}_{>0}$, and demands $d : E \rightarrow \mathbb{Z}_{>0}$, RepairWitness solves

$$\min_{S \subseteq U_p} \sum_{v \in S} w(v) \quad \text{s.t.} \quad |S \cap W_e| \geq d(e) \quad \forall e \in E.$$

Unit cost and unit demand produce the primary advisory analysis. Positive cost models execution expense; demand $f+1$ gives tolerance to the loss of any f selected releases.

Theorem 1 (NP-completeness). *The decision version of action-separating multicover is NP-complete.*

Proof. Membership in NP follows by checking all selected-edge intersections and the objective. Minimum hitting set reduces directly: map each universe element to a release, each set to an obligation, use unit costs and unit demands, and ask for a suite of size at most k . $\square$

B. Normalization and kernelization

Normalization rejects empty witness sets, nonpositive costs or demands, unknown releases, and demand exceeding witness supply. It canonicalizes identifiers and binds the complete instance with canonical JSON and SHA-256. The kernel then applies the following transformations.

Lemma 2 (Signature quotient). *If releases u and v cover identical residual obligation sets and $w(u) \leq w(v)$, an optimum exists that excludes v .*

Proof. Replace v by u in any solution containing v but not u . Every residual edge receives the same contribution, and cost does not increase. If both are present, v can be removed

unless multiplicity is required; the implementation therefore quotients only when the residual demand model establishes interchangeability for the current state. $\square$

Lemma 3 (Forced propagation). *If an obligation's remaining witness count equals its remaining demand, every remaining witness is present in every feasible completion.*

Proof. Omitting any one leaves fewer selectable witnesses than the residual demand. Selecting all forced witnesses and reducing every affected demand therefore preserves feasibility and subtracts a constant cost. $\square$

Lemma 4 (Demand-aware domination). *Suppose residual obligations e_1, e_2 satisfy $W_{e_1} \subseteq W_{e_2}$ and $d(e_1) \geq d(e_2)$. Then e_2 is redundant.*

Proof. Any selection containing at least $d(e_1)$ releases from W_{e_1} contains at least $d(e_2)$ releases from its superset W_{e_2} . $\square$

The residual bipartite graph between releases and obligations is split into connected components. Components share neither releases nor obligations.

Theorem 2 (Component additivity). *If residual components are $K_1, \dots, K_q$, then $\text{OPT}(K) = \sum_i \text{OPT}(K_i)$.*

Proof. The restriction of any global solution to each component is feasible, giving the lower bound. Conversely, the union of component-optimal solutions is feasible and has the summed cost. $\square$

C. Exact and bounded engines

Demand-state dynamic programming stores the capped coverage vector $(a_e)_{e \in E}$ with $0 \leq a_e \leq d(e)$ and therefore explores at most $\prod_e (d(e) + 1)$ states. Deterministic best-first branch-and-bound uses a greedy incumbent, branches on a tight residual obligation, and orders candidate releases by marginal cost. Its lower bound is the maximum of a disjoint-obligation packing bound, a cost-per-residual-demand bound, and component bounds. A completed frontier proves exactness; an exhausted node budget leaves a feasible upper bound and the minimum live frontier lower bound.

Theorem 3 (General certificate soundness). *If the independent checker accepts a general certificate, the selected suite satisfies the original problem, the reported objective equals its cost, the lower bound is valid, and equality of lower and upper bounds proves optimality.*

Proof. The checker independently normalizes and hashes the original instance. It recomputes the selected cost and every original edge intersection, rather than trusting the kernel. It verifies forced selections, signature representatives, domination records, and the component partition. For DP proofs it recomputes the terminal state cost; for completed branch-and-bound it checks admissible pruning and an empty improving frontier; for bounded proofs it checks the incumbent and live-frontier lower bound. Component additivity then yields global feasibility and bounds. $\square$

Algorithm 1 Independent replay of a general certificate

Require: original problem P , certificate C

- 1: $P' \leftarrow \text{NORMALIZE}(P)$; reject if $H(P') \neq C.\text{digest}$
- 2: reject unsupported schema, solver version, or proof kind
- 3: recompute selected cost; compare with $C.\text{upper}$
- 4: **for all** $e \in P'$ **do**
- 5: reject if $|C.\text{selected} \cap W_e| < d(e)$
- 6: reject any recorded edge witness outside the intersection
- 7: **end for**
- 8: recompute kernel facts and component partition
- 9: verify each component proof without invoking its producer search
- 10: reject unless component selections and bounds recompose globally
- 11: **return** accept

TABLE III
GENERAL CERTIFICATE FIELDS AND INDEPENDENT CHECKS.

Field	Producer content	Checker action
Problem digest	canonical normalized instance	recompute from original input
Selected releases	canonical unique sequence	validate membership and total cost
Edge assignments	selected witnesses per edge	recompute all intersections and demands
Kernel ledger	quotient, forced, dominated, components	independently reconstruct reductions
Bounds	component and global lower/upper	validate admissibility and recomposition
Proof kind	DP, exact BnB, bounded BnB, interval, dual	dispatch to a distinct predicate
Environment	solver/schema versions	reject unknown or incompatible versions

D. General certificate schema

A general certificate is a self-contained proof object rather than a solver log. The checker accepts only documented schema and solver versions. The principal fields are listed below.

The proof-kind field is semantically binding. A feasible selection accompanied by an exact label is rejected unless lower and upper bounds coincide and the corresponding exact proof validates. Likewise, a component cannot be omitted merely because its selected releases already cover another component in a malformed certificate; component edge sets must form a disjoint partition of the normalized obligation set.

The producer may include diagnostic counters such as explored nodes, state count, kernel sizes, and wall-clock time. These fields are not used to establish feasibility or optimality. This design prevents a fabricated node count or copied runtime from influencing acceptance.

TABLE IV
REPRESENTATIVE SEMANTIC CERTIFICATE MUTATIONS.

Mutation family	Example change	Required rejection reason
Identity	replace problem digest	instance binding mismatch
Selection	delete or insert release	feasibility or cost mismatch
Assignment	map edge to non-witness	witness-membership failure
Objective	lower/upper/cost edit	invalid bound or recomposition
Demand/cost	change one coefficient	normalized digest mismatch
Kernel	alter forced or dominated row	reconstructed reduction differs
Component	drop, merge, or overlap component	partition/recomposition failure
Proof	exact $\leftrightarrow$ bounded label	proof predicate not satisfied
Version	unsupported solver/schema	compatibility rejection
Interval	alter endpoint, deficit, insertion	ordered replay failure
Dual	perturb rational weight or cap	dual feasibility/objective failure
Evolution	stale base-suite digest	base replay/binding failure

E. Mutation and negative-control matrix

The validation campaign applies mutations after generating valid certificates. Each mutation is intended to preserve JSON syntax while violating one semantic guarantee.

The full cross-check contains 35 mutations and detects all 35. Negative tests also cover archive traversal, symlink substitution, duplicate ZIP members, oversized network responses, redirected hosts, wrong media types, malformed OR-Library rows, infeasible demands, duplicate selected releases, and authorization payload tampering in the optional source-reconstruction workflow.

III. ORDERED-RELEASE ALGORITHMS

A. Unit-cost intervals

When every witness set is contiguous in the release order, unit-cost redundant separation is solved by sorting intervals by right endpoint. For an interval whose current selected count is below demand, the algorithm inserts the rightmost available points until the deficit is zero. A Fenwick tree counts selected points in intervals; a predecessor disjoint-set finds the rightmost unselected release.

Theorem 4 (Right-endpoint optimality). *Right-endpoint insertion returns a minimum-cardinality suite for unit-cost interval obligations with integral demands.*

Proof. Process intervals by nondecreasing right endpoint. When interval I has deficit q , every feasible completion adds at least q points in I . Take an optimal completion and replace its q newly contributing points in I by the algorithm’s q rightmost available points. No processed interval loses selected points.

Any unprocessed interval that contained a replaced point has a right endpoint no earlier than I ; shifting right within I preserves membership whenever needed. Hence an optimal completion consistent with the greedy prefix exists after every step. At termination the constructed suite is optimal. $\square$

The algorithm runs in $O((m + n + |\mathcal{S}|) \log n)$ time and $O(m + n)$ auxiliary space for m obligations and n releases. Its certificate stores the release-order digest, sorted interval rows, per-step deficit, inserted indices, and selected set. Replay repeats the ordered insertions and checks all demands.

B. Positive costs and exact dual replay

For positive release costs, write the covering LP with interval-incidence matrix A :

$$\min c^T x \quad \text{s.t.} \quad Ax \geq d, \quad 0 \leq x \leq 1.$$

Every column of A^T has consecutive ones. The interval matrix is totally unimodular; adding identity rows for the upper bounds preserves total unimodularity. With integral d , an optimal extreme point is integral.

The checker does not trust floating-point equality from the producer. The producer emits the selected integral primal solution and rationalized dual variables for the obligation constraints and upper bounds. Replay verifies all inequalities with exact rational arithmetic and compares the integer primal objective with the exact dual objective.

Theorem 5 (Weighted interval optimality). *An accepted RepairWitness primal–dual certificate proves global optimality for positive-cost interval multicover.*

Proof. Primal feasibility gives an upper bound. Exact dual feasibility gives a lower bound by weak duality. Total unimodularity ensures the LP optimum equals the integer optimum. Equality of the replayed objectives therefore proves global optimality. $\square$

C. Resilience and minimum augmentation

For a fixed obligation, retaining one witness after every failure of at most f selected releases is equivalent to selecting at least $f + 1$ distinct witnesses. RepairWitness checks the witness-supply condition before solving and never counts a release twice.

For evolution, a verified base suite B may be preserved. The residual demand is $d_B(e) = \max(0, d(e) - |B \cap W_e|)$. Solving the residual instance over releases outside B minimizes added cost; a reverse-delete phase removes preserved releases only when allowed by the registered lexicographic objective. The new certificate binds the base-certificate digest and replays it before residual demands are accepted.

IV. EVALUATION EVIDENCE

A. SafetyDB historical benchmark

The historical SafetyDB benchmark is pinned to version 2021.7.17. The frozen input contains 1,246 CVE-tagged claims. Grouping by package and CVE yields 117 multi-record

TABLE V
FROZEN SAFETYDB EVIDENCE.

Measure	Value
CVE-tagged claims	1,246
Multi-record groups	117
Claim-pair edges	602
Resolved divergent / equivalent	570 / 32
Groups solved	97
Exact objective median / maximum	2 / 5
All-witness median	23
Exact-HiGHS agreement	97 / 97
One-failure-resilient groups	67
Stable-augmentation groups	72

TABLE VI
PUBLIC-OVERLAP REPLAY EVIDENCE.

Measure	Value
Redistributed source records	6
Packages	2
Paired edges	3
Affected-set equivalent edges	3
Action-divergent edges	3
Witness-count range	29–145

groups and 602 claim-pair edges. Registry acquisition records 1,319 successful, 10 empty, and 34 failed release universes. The action relation resolves 570 edges as divergent and 32 as equivalent; 97 groups contain at least one concrete divergence.

B. Public-overlap construct witness

The main paper does not use the historical SafetyDB corpus as the only evidence that affected-set comparison can miss repair-action differences. The supplementary artifact includes a small GHAD/PyPA overlap fixture with six redistributed public records, three paired edges, and frozen release universes for two PyPI packages. The verifier reparses the source records, recomputes all six normalized claims, evaluates the three paired edges, and checks that every pair is affected-set equivalent but action divergent. The fixture is intentionally not a prevalence sample: it is a source-level construct witness showing that identical affected releases can still imply different public repair actions.

Raw and normalized-record equality each misclassify 19 equivalent action edges as divergent. Affected-set equality matches all 602 labels in this corpus; the main paper explains why this empirical coincidence does not imply target-sensitive equivalence in general. The exact solver, an independent HiGHS MILP, SetCoverPy greedy, and SetCoverPy Lagrangian paths all produce feasible suites for 97 groups; exact and HiGHS objectives agree in every group. Every exact and greedy certificate replays.

The median exact suite contains two releases versus 23 in the union of all witnesses, a 90.2% median reduction. Per-edge-first selection is suboptimal in nine groups. Exact runtime has median 0.224 ms, 95th percentile 0.567 ms, and maximum 1.482 ms on the recorded environment.

TABLE VII
ALL 50 OR-LIBRARY INSTANCES. HIGHER COMPLETED/OPTIMA AND LOWER RATIOS ARE BETTER; BEST VALUES ARE BOLD.

Method	Completed	Optima	Mean ratio	Max ratio
Greedy	50	4	1.05895	1.20000
LP-guided	50	12	1.08322	1.40000
Bounded HiGHS	50	49	1.00032	1.01613

C. Controlled differential validation

The deterministic campaign separates correctness questions:

- 192 general weighted multicover instances compare exact output with exhaustive enumeration; 384 certificates replay because both exact and bounded forms are checked.
- 32 larger instances compare objective values with an independently encoded HiGHS MILP.
- 160 unit-cost interval instances compare the specialized algorithm with the general exact solver.
- 96 weighted interval instances compare with the general solver and replay exact rational primal–dual certificates.
- 64 cross-certification instances require independent proof bundles to agree and recompose.

No objective disagreement or replay failure is observed. General exact runtime has median 0.114 ms and maximum 0.555 ms; the largest explored search has 312 nodes. Greedy mean ratio is 1.019 and maximum ratio 1.333. Weighted interval certificates have median runtime 3.87 ms and maximum 922.31 ms.

A separate seeded campaign replays 1,900 certificates spanning general, interval, weighted interval, resilience, and incremental modes. A 35-field mutation cross-check changes problem digests, selected releases, bounds, costs, demands, proof kinds, component records, base-suite bindings, and interval/dual payloads. All 35 mutations are rejected. The smaller manuscript-facing campaign reports six representative semantic mutations; all six are detected.

D. OR-Library external optimization benchmark

All 50 OR-Library set-covering instances from sets 4–6 and A–E are included byte-for-byte with SHA-256 digests. Known optima are frozen independently of the solver output. Each instance is evaluated with deterministic cost-per-gain greedy plus reverse deletion, LP-guided rounding, and HiGHS MILP under a four-second per-instance budget.

The external benchmark does not establish advisory semantics; it tests the generic optimization layer on non-author-designed structures and known objectives. The bundled validator recomputes all input digests, ratios, summary statistics, CSV row counts, and the canonical report digest. The only bounded HiGHS run that does not reach a known optimum within four seconds is *scpd4*: the reported cost is 63 against known optimum 62, giving the maximum ratio 1.01613. The row is retained unchanged in the summary rather than hidden by the mean.

E. Scalability and evolution

The right-endpoint implementation is measured at 1,000, 10,000, and 100,000 releases with an equal number of obligations. At 100,000 it selects 2,942 releases, solves in 1.480 s, verifies in 0.644 s, and reaches 396.2 MiB peak resident memory. A second stress family with demands up to four and interval width up to 64 reaches 100,000 obligations in 6.91 s and verifies in 1.94 s. Every certificate is accepted by the independent checker.

The evolutionary campaign contains 96 cost-robustness, 96 one-failure-resilience, and 192 incremental cases. Cost perturbation changes the optimum in four of 96 cases. Among 56 feasible controlled resilience cases, the median additional selection is ten. Incremental updates require no new release in 107 of 192 cases and at most five in the remainder.

F. Frozen result-file schemas

Every reported experiment has a JSON summary with a schema version, fixed seed or input digest, environment tuple, row list or aggregate inventory, status, and canonical report digest. CSV files are derivative views and are checked against the JSON source. The validator does not trust stored summary statistics: it recomputes counts, means, medians, maxima, ratios, and agreement totals from row-level data where those rows are bundled.

The SafetyDB summary binds the advisory database input and release-universe digest. It records claim inventory, terminal release statuses, edge dispositions, comparator confusion counts, baseline objectives and runtimes, exact certificate status, interval applicability, resilience feasibility, incremental applicability, and total runtime. The release-universe input is not redistributed when registry terms prohibit it; its digest and deterministic acquisition recipe remain in the source manifest.

The OR-Library report contains one row per named instance. Each row binds the raw input hash, dimensions, density, known optimum, each method’s objective and ratio, runtime, and MILP status/gap. The report digest excludes only its own digest field. The artifact includes all 50 raw inputs so the report can be validated entirely offline.

The controlled validation reports are separated by purpose. One family records differential objectives, replay counts, runtime distributions, permutation invariance, and representative mutations. A second records all 35 corruption outcomes. Scalability evidence is split between the main release counts, a high-demand interval family with traced allocation, and an evolution family covering cost robustness, resilience, and augmentation.

G. Reproduction tiers

Reviewers can choose three tiers without changing code or inputs.

- 1) **Smoke** compiles modules, runs deterministic tests, executes 240 cross-oracle cases, and validates bundled summaries. It completes in minutes on one CPU.

- 2) **Full offline** additionally replays every bundled certificate, all 50 OR-Library inputs, mutation controls, and deterministic packaging. It requires no network.
- 3) **Source refresh** reacquires pinned public advisory repositories or mutable registry responses, records new receipts, and produces a separate dated reconstruction. A refresh never overwrites frozen benchmark evidence.

The first two tiers are sufficient to reproduce all paper figures and tables. Source refresh is provided to study temporal evolution, not as a hidden dependency of the submitted results.

H. Dependency and environment control

The implementation requires Python 3.11 or newer and pins parser dependencies that affect semantics. NumPy and SciPy are bounded by major version; exact certificate replay itself uses Python integers and rational arithmetic and does not require an external MILP solver. Tests invoke HiGHS through SciPy only for independent objective comparison. The manuscript uses the unmodified IEEE conference class, TikZ, PGFPlots, booktabs, and algorithmic pseudocode.

The replay harness records interpreter, platform, package versions, test count, and evidence digests. Network access is disabled for offline replay. When source acquisition is requested, the network layer restricts scheme, port, host allowlist, redirects, media type, declared and actual length, response size, and atomic destination replacement. Every response receipt includes request URL, terminal URL, status, headers used for validation, byte count, and SHA-256.

V. ARTIFACT AND REPRODUCTION CONTRACT

A. Reproduction entry point

The offline entry point compiles the implementation, executes the deterministic test suite, runs cross-oracle method smoke checks, validates every bundled result file, performs the anonymity and repository audit, creates the combined and code-only archives twice, and compares their bytes. The in-tree attestation records the pre-package subject state; a separate run-local PASS report is written outside the project tree only after packaging and clean replay also succeed.

B. Deterministic packaging

Publishable files are enumerated in canonical path order. ZIP members use a fixed timestamp, fixed Unix permissions, UTF-8 paths, and maximum deflate compression. Symlinks, unsafe paths, caches, build outputs, raw logs, local absolute paths, credentials, and author-identity patterns are rejected. The combined archive contains exactly the manuscript surface and the reproduction artifact; the code archive contains only the reproduction artifact.

Safe extraction rejects path traversal before writing bytes. Clean replay extracts into a fresh temporary directory, reruns the offline entry point, and compares tree and archive digests. The release manifest records file count, byte count, individual SHA-256 values, and archive SHA-256 values.

C. Result-to-paper traceability

The result validator checks four evidence families. For OR-Library it recomputes 50 input hashes, approximation ratios, summary statistics, CSV rows, and report digest. For SafetyDB it checks disposition totals, group counts, certificate validity, and exact-MILP agreement. For the public-overlap fixture it reparses included GHAD/PyPA records and recomputes affected-set and action decisions. For controlled validation it checks certificate counts, mutation detection, and scalability rows. The paper’s plot data are generated from frozen text summaries rather than external screenshots.

The reference ledger contains one row per cited BibTeX key and records either a stable DOI/ISBN/arXiv locator, an official URL, or manually checked venue metadata. The audit rejects unused bibliography entries and missing ledger keys. No screenshot or externally edited figure appears in the paper; all figures are generated by TikZ or PGFPlots from frozen text data.

D. Anonymity boundary

Project-authored files contain no author names, affiliations, email addresses, personal repository URLs, local home directories, or conversation material. Commit metadata is outside the distributed ZIP. Upstream benchmark and advisory bytes retain their public provenance; such text is external evidence rather than project authorship.

VI. INDEPENDENT CHECKER SOUNDNESS

A. Acceptance sequence

The checker is intentionally smaller than the producer and accepts no producer-side cache, search frontier, or mutable global state. It processes a certificate in the following order. First, it canonicalizes the release identifiers, costs, demands, witness memberships, and algorithm parameters, then compares the resulting problem digest with the certificate. Second, it rejects duplicate or unknown releases and recomputes every selected release’s contribution to every obligation. Third, it recomputes total cost and checks the claimed upper bound. Fourth, it dispatches only on a closed set of proof kinds. General exact certificates must provide a component partition and a lower bound for every component; bounded certificates must provide a valid incumbent and a frontier-derived lower bound; interval certificates must replay each right-endpoint insertion; weighted interval certificates must satisfy both primal feasibility and the exact rational dual constraints. Finally, the checker recomposes component values and verifies that the global status follows from the checked bounds.

The order is security-relevant. Checking a proof payload before binding the instance would permit a valid proof to be transplanted to different obligations. Checking only aggregate coverage would permit the producer to substitute an unlisted release. Trusting a stored objective would permit cost changes after search. RepairWitness therefore derives all three quantities—instance identity, feasibility, and objective—from canonical input bytes before interpreting any optimality claim.

Theorem 6 (Checker soundness). *Assume collision resistance of SHA-256 and correctness of integer and rational arithmetic. If the RepairWitness checker accepts a certificate with selected suite S , reported cost C , lower bound L , and status OPTIMAL, then S satisfies every certified demand, $C = \sum_{r \in S} c(r)$, every feasible suite has cost at least L , and $C = L$. For status BOUNDED, the same feasibility and cost statements hold and $L \leq \text{OPT} \leq C$.*

Proof. Digest verification binds the certificate to one canonical instance. The checker enumerates every obligation and counts selected members of its witness set, so acceptance implies primal feasibility. It independently sums the bound costs of distinct selected releases, establishing the reported upper bound. For each accepted proof kind, the corresponding local checker establishes a lower bound: exhaustive leaf closure and frontier minima for search certificates, exact dynamic-program values for state certificates, or weak duality for rational primal–dual certificates. Component partitions are checked for disjointness and completeness, so local lower bounds sum to a global lower bound. An OPTIMAL status is accepted only when the recomputed upper and lower bounds are equal; a BOUNDED status records both without equality. $\square$

B. Completeness of emitted certificates

Soundness does not require the checker to accept every possible proof of optimality, only every proof kind emitted by the producer. The exact general solver emits a complete certificate when forced propagation, signature quotienting, domination, component decomposition, dynamic programming, or branch-and-bound closes every component. If a search budget expires, the producer emits a bounded certificate rather than relabeling the incumbent as optimal. The unit-cost interval algorithm emits one insertion step for each right endpoint whose residual demand is positive; replaying the predecessor disjoint-set choices reconstructs the selected set. The weighted interval path emits a primal solution and an exact rational dual solution over the consecutive-ones formulation. Equality of their objectives establishes optimality without rerunning the producer.

This design also localizes unsupported cases. If interval recognition fails, the instance remains valid and is routed to the general solver. If a component exceeds an exact budget, other components may still be exact while the global certificate remains bounded. If an external MILP backend is absent, certificate replay still succeeds because the checker uses no floating-point optimizer. Thus missing optional machinery reduces proof strength or runtime, not semantic correctness.

C. Mutation adequacy and negative controls

The mutation campaign targets the boundaries that a plausible but incomplete checker might omit. Mutations alter the problem digest, selected release list, release cost, edge demand, witness membership, proof kind, component partition, component objective, global bounds, interval step, rational dual variable, preserved base suite, and augmentation digest. A mutation is counted as detected only when the independent

checker rejects the modified object; parser exceptions and producer-side assertions do not count. All 35 cross-check mutations are rejected, as are the six representative mutations reported in the main paper.

Mutation success is not presented as a probability that the checker is bug-free. Its role is structural: each security-relevant field has at least one negative control demonstrating that the field participates in acceptance. Differential campaigns provide a separate axis of evidence by comparing accepted objectives with exhaustive enumeration, generic exact search, and HiGHS. Permutation tests then verify that canonical ordering, rather than input accident, determines the emitted certificate.

D. Release replay and non-self-reference

The release workflow treats reproducibility as another certificate problem. Publishable paths are enumerated canonically; forbidden caches, symlinks, traversal paths, generated logs, local paths, and identity strings cause failure. ZIP members use fixed timestamps and permissions. Two independently generated archives must be byte-identical. A fresh extractor rejects unsafe member names, writes into a temporary root, audits the extracted tree, and runs the reviewer smoke checks there.

The transport archive hash is distributed beside the archive rather than embedded inside it, avoiding a self-referential digest. The in-archive release attestation instead binds the audited subject-tree digest, PDF hashes, page counts, reference count, and benchmark inventories; the evaluator-local packaged-release attestation binds archive byte equality and clean extraction replay. This separation lets a reviewer verify both the contents and the transport object without trusting a circular manifest.

The checker and release workflow establish internal reproducibility, not the truth of public advisories. Upstream errors can survive perfect replay. The contribution is that such errors, source updates, semantic choices, and optimization choices remain separately identifiable rather than collapsing into an opaque result.

VII. ADDITIONAL VALIDITY NOTES

SafetyDB is historical and PyPI-specific. It demonstrates real duplicate-claim behavior, but its 94.7% divergent-edge share is not presented as a universal prevalence estimate. The public-overlap fixture demonstrates a target-sensitive construct on real public source records, but it is deliberately too small to estimate frequency. OR-Library validates optimization generality, not security-advisory semantics. Controlled instances validate implementation and proof checking, not upstream advisory correctness. These layers are intentionally complementary.

RepairWitness’s construct is a source-relative executable action. A fixed anchor is not relabeled as a safe release. Unknown outcomes remain in coverage and blocker reporting but never become witnesses. Shared lineage is not treated as independent curator consensus. These constraints narrow

the substantive claim while allowing the formal and empirical evidence to be checked exactly.

VIII. TRACE PARTITION BOUNDS

The interval path is recognized from realized action traces, but the same representation also gives a quantitative description when one interval is insufficient. Let $b(a)$ denote the number of maximal constant blocks in an action trace a over a totally ordered release universe. A block boundary occurs exactly where the action changes between adjacent releases.

Theorem 7 (Common-refinement bound). *For two piecewise-constant action traces a_1 and a_2 , their concrete disagreement set is a union of at most $b(a_1) + b(a_2) - 1$ disjoint release intervals.*

Proof. The boundaries of a_1 and a_2 jointly partition the release universe into at most $(b(a_1) - 1) + (b(a_2) - 1) + 1 = b(a_1) + b(a_2) - 1$ common-refinement blocks. Both actions are constant on every such block, so equality or disagreement is constant there as well. The disagreement set is therefore a union of a subset of these ordered blocks; merging adjacent selected blocks cannot increase their number. $\square$

The bound is deliberately conservative. Single-boundary upgrade claims have two blocks each, yet the stronger theorem in the main paper shows that their disagreement set is at most one interval. The common-refinement theorem applies to richer traces containing withdrawn segments, explicit unaffected islands, alternative mitigations, or changes from a public target to repair-without-target. It provides a compact diagnostic even when the one-interval optimizer is inapplicable: a checker can report the number and endpoints of disagreement runs before routing the obligation to the general multicover solver.

This view prevents an unsafe syntactic shortcut. An advisory range with several clauses does not necessarily yield several semantic intervals, because empty gaps in the public release history disappear after projection. Conversely, a single prose record may induce several intervals when its action changes across release-specific exceptions. RepairWitness therefore computes blocks from the digest-bound release universe and normalized action trace rather than from clause count.

Proposition 2 (Linear certificate representation for unit-cost intervals). *For m interval obligations over n ordered releases, the right-endpoint producer can represent its replay certificate with one canonical problem record, one obligation record per interval, and one insertion record per selected release. Excluding the encoded identifiers themselves, the number of records is $O(m + |S|)$ and hence $O(m + n)$.*

Proof. Each obligation is identified by its two endpoints and demand, so it contributes one record. The producer selects any release at most once; all obligations covered by that release are recomputed by the checker and need not be enumerated in the insertion record. Because $|S| \leq n$, the record count is linear in the input dimensions. $\square$

The linear representation is important for the six-figure experiment. It keeps the evidence object proportional to the release sequence and obligation inventory rather than to the potentially quadratic edge–release incidence matrix. The checker still derives incidence from endpoints, so compression does not reduce scrutiny. For general noninterval witness sets, the artifact uses canonical sparse membership lists and component certificates instead.

The partition bound also clarifies the meaning of a failed interval-recognition check. Failure is not an error and does not discard an advisory edge. It establishes only that the edge requires more than one realized disagreement run or contains a nonconcrete action. The first case is solved by the general certified path; the second is retained as an abstention. Thus specialization changes computational strategy without changing the semantic denominator.

References used in this supplement are listed in the main paper.